

Introduction to Machine Learning

Second
Edition

Ethem Alpaydm

The MIT Press
Cambridge, Massachusetts
London, England

© 2010 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

For information about special quantity discounts, please email
special_sales@mitpress.mit.edu.

Typeset in 10/13 Lucida Bright by the author using $\text{\LaTeX}$ 2 ϵ .
Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Information

Alpaydin, Ethem.

Introduction to machine learning / Ethem Alpaydin. — 2nd ed.

p. cm.

Includes bibliographical references and index.

ISBN 978-0-262-01243-0 (hardcover : alk. paper)

1. Machine learning. I. Title

Q325.5.A46 2010

006.3'1—dc22

2009013169

CIP

10 9 8 7 6 5 4 3 2 1

Brief Contents

1	<i>Introduction</i>	1
2	<i>Supervised Learning</i>	21
3	<i>Bayesian Decision Theory</i>	47
4	<i>Parametric Methods</i>	61
5	<i>Multivariate Methods</i>	87
6	<i>Dimensionality Reduction</i>	109
7	<i>Clustering</i>	143
8	<i>Nonparametric Methods</i>	163
9	<i>Decision Trees</i>	185
10	<i>Linear Discrimination</i>	209
11	<i>Multilayer Perceptrons</i>	233
12	<i>Local Models</i>	279
13	<i>Kernel Machines</i>	309
14	<i>Bayesian Estimation</i>	341
15	<i>Hidden Markov Models</i>	363
16	<i>Graphical Models</i>	387
17	<i>Combining Multiple Learners</i>	419
18	<i>Reinforcement Learning</i>	447
19	<i>Design and Analysis of Machine Learning Experiments</i>	475
A	<i>Probability</i>	517

Contents

Series Foreword xvii

Figures xix

Tables xxix

Preface xxxi

Acknowledgments xxxiii

Notes for the Second Edition xxxv

Notations xxxix

1 *Introduction* 1

- 1.1 What Is Machine Learning? 1
- 1.2 Examples of Machine Learning Applications 4
 - 1.2.1 Learning Associations 4
 - 1.2.2 Classification 5
 - 1.2.3 Regression 9
 - 1.2.4 Unsupervised Learning 11
 - 1.2.5 Reinforcement Learning 13
- 1.3 Notes 14
- 1.4 Relevant Resources 16
- 1.5 Exercises 18
- 1.6 References 19

2 *Supervised Learning* 21

- 2.1 Learning a Class from Examples 21

2.2	Vapnik-Chervonenkis (VC) Dimension	27
2.3	Probably Approximately Correct (PAC) Learning	29
2.4	Noise	30
2.5	Learning Multiple Classes	32
2.6	Regression	34
2.7	Model Selection and Generalization	37
2.8	Dimensions of a Supervised Machine Learning Algorithm	41
2.9	Notes	42
2.10	Exercises	43
2.11	References	44
3	<i>Bayesian Decision Theory</i>	47
3.1	Introduction	47
3.2	Classification	49
3.3	Losses and Risks	51
3.4	Discriminant Functions	53
3.5	Utility Theory	54
3.6	Association Rules	55
3.7	Notes	58
3.8	Exercises	58
3.9	References	59
4	<i>Parametric Methods</i>	61
4.1	Introduction	61
4.2	Maximum Likelihood Estimation	62
4.2.1	Bernoulli Density	63
4.2.2	Multinomial Density	64
4.2.3	Gaussian (Normal) Density	64
4.3	Evaluating an Estimator: Bias and Variance	65
4.4	The Bayes' Estimator	66
4.5	Parametric Classification	69
4.6	Regression	73
4.7	Tuning Model Complexity: Bias/Variance Dilemma	76
4.8	Model Selection Procedures	80
4.9	Notes	84
4.10	Exercises	84
4.11	References	85
5	<i>Multivariate Methods</i>	87
5.1	Multivariate Data	87

5.2	Parameter Estimation	88
5.3	Estimation of Missing Values	89
5.4	Multivariate Normal Distribution	90
5.5	Multivariate Classification	94
5.6	Tuning Complexity	99
5.7	Discrete Features	102
5.8	Multivariate Regression	103
5.9	Notes	105
5.10	Exercises	106
5.11	References	107
6	<i>Dimensionality Reduction</i>	109
6.1	Introduction	109
6.2	Subset Selection	110
6.3	Principal Components Analysis	113
6.4	Factor Analysis	120
6.5	Multidimensional Scaling	125
6.6	Linear Discriminant Analysis	128
6.7	Isomap	133
6.8	Locally Linear Embedding	135
6.9	Notes	138
6.10	Exercises	139
6.11	References	140
7	<i>Clustering</i>	143
7.1	Introduction	143
7.2	Mixture Densities	144
7.3	<i>k</i> -Means Clustering	145
7.4	Expectation-Maximization Algorithm	149
7.5	Mixtures of Latent Variable Models	154
7.6	Supervised Learning after Clustering	155
7.7	Hierarchical Clustering	157
7.8	Choosing the Number of Clusters	158
7.9	Notes	160
7.10	Exercises	160
7.11	References	161
8	<i>Nonparametric Methods</i>	163
8.1	Introduction	163
8.2	Nonparametric Density Estimation	165

8.2.1	Histogram Estimator	165
8.2.2	Kernel Estimator	167
8.2.3	k -Nearest Neighbor Estimator	168
8.3	Generalization to Multivariate Data	170
8.4	Nonparametric Classification	171
8.5	Condensed Nearest Neighbor	172
8.6	Nonparametric Regression: Smoothing Models	174
8.6.1	Running Mean Smoother	175
8.6.2	Kernel Smoother	176
8.6.3	Running Line Smoother	177
8.7	How to Choose the Smoothing Parameter	178
8.8	Notes	180
8.9	Exercises	181
8.10	References	182
9	<i>Decision Trees</i>	185
9.1	Introduction	185
9.2	Univariate Trees	187
9.2.1	Classification Trees	188
9.2.2	Regression Trees	192
9.3	Pruning	194
9.4	Rule Extraction from Trees	197
9.5	Learning Rules from Data	198
9.6	Multivariate Trees	202
9.7	Notes	204
9.8	Exercises	207
9.9	References	207
10	<i>Linear Discrimination</i>	209
10.1	Introduction	209
10.2	Generalizing the Linear Model	211
10.3	Geometry of the Linear Discriminant	212
10.3.1	Two Classes	212
10.3.2	Multiple Classes	214
10.4	Pairwise Separation	216
10.5	Parametric Discrimination Revisited	217
10.6	Gradient Descent	218
10.7	Logistic Discrimination	220
10.7.1	Two Classes	220

10.7.2 Multiple Classes	224
10.8 Discrimination by Regression	228
10.9 Notes	230
10.10 Exercises	230
10.11 References	231
11 Multilayer Perceptrons	233
11.1 Introduction	233
11.1.1 Understanding the Brain	234
11.1.2 Neural Networks as a Paradigm for Parallel Processing	235
11.2 The Perceptron	237
11.3 Training a Perceptron	240
11.4 Learning Boolean Functions	243
11.5 Multilayer Perceptrons	245
11.6 MLP as a Universal Approximator	248
11.7 Backpropagation Algorithm	249
11.7.1 Nonlinear Regression	250
11.7.2 Two-Class Discrimination	252
11.7.3 Multiclass Discrimination	254
11.7.4 Multiple Hidden Layers	256
11.8 Training Procedures	256
11.8.1 Improving Convergence	256
11.8.2 Overtraining	257
11.8.3 Structuring the Network	258
11.8.4 Hints	261
11.9 Tuning the Network Size	263
11.10 Bayesian View of Learning	266
11.11 Dimensionality Reduction	267
11.12 Learning Time	270
11.12.1 Time Delay Neural Networks	270
11.12.2 Recurrent Networks	271
11.13 Notes	272
11.14 Exercises	274
11.15 References	275
12 Local Models	279
12.1 Introduction	279
12.2 Competitive Learning	280

12.2.1	Online k -Means	280
12.2.2	Adaptive Resonance Theory	285
12.2.3	Self-Organizing Maps	286
12.3	Radial Basis Functions	288
12.4	Incorporating Rule-Based Knowledge	294
12.5	Normalized Basis Functions	295
12.6	Competitive Basis Functions	297
12.7	Learning Vector Quantization	300
12.8	Mixture of Experts	300
12.8.1	Cooperative Experts	303
12.8.2	Competitive Experts	304
12.9	Hierarchical Mixture of Experts	304
12.10	Notes	305
12.11	Exercises	306
12.12	References	307
13	Kernel Machines	309
13.1	Introduction	309
13.2	Optimal Separating Hyperplane	311
13.3	The Nonseparable Case: Soft Margin Hyperplane	315
13.4	ν -SVM	318
13.5	Kernel Trick	319
13.6	Vectorial Kernels	321
13.7	Defining Kernels	324
13.8	Multiple Kernel Learning	325
13.9	Multiclass Kernel Machines	327
13.10	Kernel Machines for Regression	328
13.11	One-Class Kernel Machines	333
13.12	Kernel Dimensionality Reduction	335
13.13	Notes	337
13.14	Exercises	338
13.15	References	339
14	Bayesian Estimation	341
14.1	Introduction	341
14.2	Estimating the Parameter of a Distribution	343
14.2.1	Discrete Variables	343
14.2.2	Continuous Variables	345
14.3	Bayesian Estimation of the Parameters of a Function	348

14.3.1	Regression	348
14.3.2	The Use of Basis/Kernel Functions	352
14.3.3	Bayesian Classification	353
14.4	Gaussian Processes	356
14.5	Notes	359
14.6	Exercises	360
14.7	References	361
15	<i>Hidden Markov Models</i>	363
15.1	Introduction	363
15.2	Discrete Markov Processes	364
15.3	Hidden Markov Models	367
15.4	Three Basic Problems of HMMs	369
15.5	Evaluation Problem	369
15.6	Finding the State Sequence	373
15.7	Learning Model Parameters	375
15.8	Continuous Observations	378
15.9	The HMM with Input	379
15.10	Model Selection in HMM	380
15.11	Notes	382
15.12	Exercises	383
15.13	References	384
16	<i>Graphical Models</i>	387
16.1	Introduction	387
16.2	Canonical Cases for Conditional Independence	389
16.3	Example Graphical Models	396
16.3.1	Naive Bayes' Classifier	396
16.3.2	Hidden Markov Model	398
16.3.3	Linear Regression	401
16.4	d-Separation	402
16.5	Belief Propagation	402
16.5.1	Chains	403
16.5.2	Trees	405
16.5.3	Polytrees	407
16.5.4	Junction Trees	409
16.6	Undirected Graphs: Markov Random Fields	410
16.7	Learning the Structure of a Graphical Model	413
16.8	Influence Diagrams	414

16.9	Notes	414
16.10	Exercises	417
16.11	References	417
17	<i>Combining Multiple Learners</i>	419
17.1	Rationale	419
17.2	Generating Diverse Learners	420
17.3	Model Combination Schemes	423
17.4	Voting	424
17.5	Error-Correcting Output Codes	427
17.6	Bagging	430
17.7	Boosting	431
17.8	Mixture of Experts Revisited	434
17.9	Stacked Generalization	435
17.10	Fine-Tuning an Ensemble	437
17.11	Cascading	438
17.12	Notes	440
17.13	Exercises	442
17.14	References	443
18	<i>Reinforcement Learning</i>	447
18.1	Introduction	447
18.2	Single State Case: K -Armed Bandit	449
18.3	Elements of Reinforcement Learning	450
18.4	Model-Based Learning	453
18.4.1	Value Iteration	453
18.4.2	Policy Iteration	454
18.5	Temporal Difference Learning	454
18.5.1	Exploration Strategies	455
18.5.2	Deterministic Rewards and Actions	456
18.5.3	Nondeterministic Rewards and Actions	457
18.5.4	Eligibility Traces	459
18.6	Generalization	461
18.7	Partially Observable States	464
18.7.1	The Setting	464
18.7.2	Example: The Tiger Problem	465
18.8	Notes	470
18.9	Exercises	472
18.10	References	473

19 Design and Analysis of Machine Learning Experiments	475
19.1 Introduction	475
19.2 Factors, Response, and Strategy of Experimentation	478
19.3 Response Surface Design	481
19.4 Randomization, Replication, and Blocking	482
19.5 Guidelines for Machine Learning Experiments	483
19.6 Cross-Validation and Resampling Methods	486
19.6.1 K -Fold Cross-Validation	487
19.6.2 5×2 Cross-Validation	488
19.6.3 Bootstrapping	489
19.7 Measuring Classifier Performance	489
19.8 Interval Estimation	493
19.9 Hypothesis Testing	496
19.10 Assessing a Classification Algorithm's Performance	498
19.10.1 Binomial Test	499
19.10.2 Approximate Normal Test	500
19.10.3 t Test	500
19.11 Comparing Two Classification Algorithms	501
19.11.1 McNemar's Test	501
19.11.2 K -Fold Cross-Validated Paired t Test	501
19.11.3 5×2 cv Paired t Test	502
19.11.4 5×2 cv Paired F Test	503
19.12 Comparing Multiple Algorithms: Analysis of Variance	504
19.13 Comparison over Multiple Datasets	508
19.13.1 Comparing Two Algorithms	509
19.13.2 Multiple Algorithms	511
19.14 Notes	512
19.15 Exercises	513
19.16 References	514
A Probability	517
A.1 Elements of Probability	517
A.1.1 Axioms of Probability	518
A.1.2 Conditional Probability	518
A.2 Random Variables	519
A.2.1 Probability Distribution and Density Functions	519
A.2.2 Joint Distribution and Density Functions	520
A.2.3 Conditional Distributions	520
A.2.4 Bayes' Rule	521

	A.2.5	Expectation	521
	A.2.6	Variance	522
	A.2.7	Weak Law of Large Numbers	523
A.3	Special	Random Variables	523
	A.3.1	Bernoulli Distribution	523
	A.3.2	Binomial Distribution	524
	A.3.3	Multinomial Distribution	524
	A.3.4	Uniform Distribution	524
	A.3.5	Normal (Gaussian) Distribution	525
	A.3.6	Chi-Square Distribution	526
	A.3.7	t Distribution	527
	A.3.8	F Distribution	527
A.4	References		527
<i>Index</i>			529

Series Foreword

The goal of building systems that can adapt to their environments and learn from their experience has attracted researchers from many fields, including computer science, engineering, mathematics, physics, neuroscience, and cognitive science. Out of this research has come a wide variety of learning techniques that are transforming many industrial and scientific fields. Recently, several research communities have converged on a common set of issues surrounding supervised, semi-supervised, unsupervised, and reinforcement learning problems. The MIT Press Series on Adaptive Computation and Machine Learning seeks to unify the many diverse strands of machine learning research and to foster high-quality research and innovative applications.

The MIT Press is extremely pleased to publish this second edition of Ethem Alpaydın's introductory textbook. This book presents a readable and concise introduction to machine learning that reflects these diverse research strands while providing a unified treatment of the field. The book covers all of the main problem formulations and introduces the most important algorithms and techniques encompassing methods from computer science, neural computation, information theory, and statistics. The second edition expands and updates coverage of several areas, particularly kernel machines and graphical models, that have advanced rapidly over the past five years. This updated work continues to be a compelling textbook for introductory courses in machine learning at the undergraduate and beginning graduate level.

Figures

1.1	Example of a training dataset where each circle corresponds to one data instance with input values in the corresponding axes and its sign indicates the class.	6
1.2	A training dataset of used cars and the function fitted.	10
2.1	Training set for the class of a “family car.”	22
2.2	Example of a hypothesis class.	23
2.3	C is the actual class and h is our induced hypothesis.	25
2.4	S is the most specific and G is the most general hypothesis.	26
2.5	We choose the hypothesis with the largest margin, for best separation.	27
2.6	An axis-aligned rectangle can shatter four points.	28
2.7	The difference between h and C is the sum of four rectangular strips, one of which is shaded.	30
2.8	When there is noise, there is not a simple boundary between the positive and negative instances, and zero misclassification error may not be possible with a simple hypothesis.	31
2.9	There are three classes: family car, sports car, and luxury sedan.	33
2.10	Linear, second-order, and sixth-order polynomials are fitted to the same set of points.	36
2.11	A line separating positive and negative instances.	44
3.1	Example of decision regions and decision boundaries.	54

4.1	θ is the parameter to be estimated.	67
4.2	(a) Likelihood functions and (b) posteriors with equal priors for two classes when the input is one-dimensional.	71
4.3	(a) Likelihood functions and (b) posteriors with equal priors for two classes when the input is one-dimensional.	72
4.4	Regression assumes 0 mean Gaussian noise added to the model; here, the model is linear.	74
4.5	(a) Function, $f(x) = 2 \sin(1.5x)$, and one noisy ($\mathcal{N}(0, 1)$) dataset sampled from the function.	78
4.6	In the same setting as that of figure 4.5, using one hundred models instead of five, bias, variance, and error for polynomials of order 1 to 5.	79
4.7	In the same setting as that of figure 4.5, training and validation sets (each containing 50 instances) are generated.	81
4.8	In the same setting as that of figure 4.5, polynomials of order 1 to 4 are fitted.	83
5.1	Bivariate normal distribution.	91
5.2	Isoprobability contour plot of the bivariate normal distribution.	92
5.3	Classes have different covariance matrices.	96
5.4	Covariances may be arbitrary but shared by both classes.	97
5.5	All classes have equal, diagonal covariance matrices, but variances are not equal.	98
5.6	All classes have equal, diagonal covariance matrices of equal variances on both dimensions.	99
5.7	Different cases of the covariance matrices fitted to the same data lead to different boundaries.	101
6.1	Principal components analysis centers the sample and then rotates the axes to line up with the directions of highest variance.	115
6.2	(a) Scree graph. (b) Proportion of variance explained is given for the Optdigits dataset from the UCI Repository.	117
6.3	Optdigits data plotted in the space of two principal components.	118
6.4	Principal components analysis generates new variables that are linear combinations of the original input variables.	121

6.5	Factors are independent unit normals that are stretched, rotated, and translated to make up the inputs.	122
6.6	Map of Europe drawn by MDS.	126
6.7	Two-dimensional, two-class data projected on $\mathbf{w}$.	129
6.8	Optdigits data plotted in the space of the first two dimensions found by LDA.	132
6.9	Geodesic distance is calculated along the manifold as opposed to the Euclidean distance that does not use this information.	134
6.10	Local linear embedding first learns the constraints in the original space and next places the points in the new space respecting those constraints.	136
7.1	Given $\mathbf{x}$, the encoder sends the index of the closest code word and the decoder generates the code word with the received index as $\mathbf{x}'$.	147
7.2	Evolution of k -means.	148
7.3	k -means algorithm.	149
7.4	Data points and the fitted Gaussians by EM, initialized by one k -means iteration of figure 7.2.	153
7.5	A two-dimensional dataset and the dendrogram showing the result of single-link clustering is shown.	159
8.1	Histograms for various bin lengths.	166
8.2	Naive estimate for various bin lengths.	167
8.3	Kernel estimate for various bin lengths.	168
8.4	k -nearest neighbor estimate for various k values.	169
8.5	Dotted lines are the Voronoi tessellation and the straight line is the class discriminant.	173
8.6	Condensed nearest neighbor algorithm.	174
8.7	Regressograms for various bin lengths. '×' denote data points.	175
8.8	Running mean smooth for various bin lengths.	176
8.9	Kernel smooth for various bin lengths.	177
8.10	Running line smooth for various bin lengths.	178
8.11	Kernel estimate for various bin lengths for a two-class problem.	179
8.12	Regressograms with linear fits in bins for various bin lengths.	182

9.1	Example of a dataset and the corresponding decision tree.	186
9.2	Entropy function for a two-class problem.	189
9.3	Classification tree construction.	191
9.4	Regression tree smooths for various values of θ_r .	195
9.5	Regression trees implementing the smooths of figure 9.4 for various values of θ_r .	196
9.6	Example of a (hypothetical) decision tree.	197
9.7	Ripper algorithm for learning rules.	200
9.8	Example of a linear multivariate decision tree.	203
10.1	In the two-dimensional case, the linear discriminant is a line that separates the examples from two classes.	213
10.2	The geometric interpretation of the linear discriminant.	214
10.3	In linear classification, each hyperplane H_i separates the examples of C_i from the examples of all other classes.	215
10.4	In pairwise linear separation, there is a separate hyperplane for each pair of classes.	216
10.5	The logistic, or sigmoid, function.	219
10.6	Logistic discrimination algorithm implementing gradient descent for the single output case with two classes.	222
10.7	For a univariate two-class problem (shown with 'o' and 'x'), the evolution of the line $wx + w_0$ and the sigmoid output after 10, 100, and 1,000 iterations over the sample.	223
10.8	Logistic discrimination algorithm implementing gradient descent for the case with $K > 2$ classes.	226
10.9	For a two-dimensional problem with three classes, the solution found by logistic discrimination.	226
10.10	For the same example in figure 10.9, the linear discriminants (top), and the posterior probabilities after the softmax (bottom).	227
11.1	Simple perceptron.	237
11.2	K parallel perceptrons.	239
11.3	Perceptron training algorithm implementing stochastic online gradient descent for the case with $K > 2$ classes.	243
11.4	The perceptron that implements AND and its geometric interpretation.	244
11.5	XOR problem is not linearly separable.	245
11.6	The structure of a multilayer perceptron.	247

11.7	The multilayer perceptron that solves the XOR problem.	249
11.8	Sample training data shown as '+', where $\mathbf{x}^t \sim U(-0.5, 0.5)$, and $y^t = f(\mathbf{x}^t) + \mathcal{N}(0, 0.1)$.	252
11.9	The mean square error on training and validation sets as a function of training epochs.	253
11.10	(a) The hyperplanes of the hidden unit weights on the first layer, (b) hidden unit outputs, and (c) hidden unit outputs multiplied by the weights on the second layer.	254
11.11	Backpropagation algorithm for training a multilayer perceptron for regression with K outputs.	255
11.12	As complexity increases, training error is fixed but the validation error starts to increase and the network starts to overfit.	259
11.13	As training continues, the validation error starts to increase and the network starts to overfit.	259
11.14	A structured MLP.	260
11.15	In weight sharing, different units have connections to different inputs but share the same weight value (denoted by line type).	261
11.16	The identity of the object does not change when it is translated, rotated, or scaled.	262
11.17	Two examples of constructive algorithms.	265
11.18	Optdigits data plotted in the space of the two hidden units of an MLP trained for classification.	268
11.19	In the autoassociator, there are as many outputs as there are inputs and the desired outputs are the inputs.	269
11.20	A time delay neural network.	271
11.21	Examples of MLP with partial recurrency.	272
11.22	Backpropagation through time.	273
12.1	Shaded circles are the centers and the empty circle is the input instance.	282
12.2	Online k -means algorithm.	283
12.3	The winner-take-all competitive neural network, which is a network of k perceptrons with recurrent connections at the output.	284
12.4	The distance from $\mathbf{x}^a$ to the closest center is less than the vigilance value ρ and the center is updated as in online k -means.	285

12.5	In the SOM, not only the closest unit but also its neighbors, in terms of indices, are moved toward the input.	287
12.6	The one-dimensional form of the bell-shaped function used in the radial basis function network.	289
12.7	The difference between local and distributed representations.	290
12.8	The RBF network where p_h are the hidden units using the bell-shaped activation function.	292
12.9	(-) Before and (- -) after normalization for three Gaussians whose centers are denoted by “*”.	296
12.10	The mixture of experts can be seen as an RBF network where the second-layer weights are outputs of linear models.	301
12.11	The mixture of experts can be seen as a model for combining multiple models.	302
13.1	For a two-class problem where the instances of the classes are shown by plus signs and dots, the thick line is the boundary and the dashed lines define the margins on either side.	314
13.2	In classifying an instance, there are four possible cases.	316
13.3	Comparison of different loss functions for $r^t = 1$.	318
13.4	The discriminant and margins found by a polynomial kernel of degree 2.	322
13.5	The boundary and margins found by the Gaussian kernel with different spread values, s^2 .	323
13.6	Quadratic and ϵ -sensitive error functions.	329
13.7	The fitted regression line to data points shown as crosses and the ϵ -tube are shown ($C = 10, \epsilon = 0.25$).	331
13.8	The fitted regression line and the ϵ -tube using a quadratic kernel are shown ($C = 10, \epsilon = 0.25$).	332
13.9	The fitted regression line and the ϵ -tube using a Gaussian kernel with two different spreads are shown ($C = 10, \epsilon = 0.25$).	332
13.10	One-class support vector machine places the smoothest boundary (here using a linear kernel, the circle with the smallest radius) that encloses as much of the instances as possible.	334
13.11	One-class support vector machine using a Gaussian kernel with different spreads.	336

13.12	Instead of using a quadratic kernel in the original space (a), we can use kernel PCA on the quadratic kernel values to map to a two-dimensional new space where we use a linear discriminant (b); these two dimensions (out of five) explain 80 percent of the variance.	337
14.1	The generative graphical model.	342
14.2	Plots of beta distributions for different sets of (α, β) .	346
14.3	20 data points are drawn from $p(x) \sim \mathcal{N}(6, 1.5^2)$, prior is $p(\mu) \sim \mathcal{N}(4, 0.8^2)$, and posterior is then $p(\mu \mathcal{X}) \sim \mathcal{N}(5.7, 0.3^2)$.	347
14.4	Bayesian linear regression for different values of α and β .	351
14.5	Bayesian regression using kernels with one standard deviation error bars.	354
14.6	Gaussian process regression with one standard deviation error bars.	357
14.7	Gaussian process regression using a Gaussian kernel with $s^2 = 0.5$ and varying number of training data.	359
15.1	Example of a Markov model with three states.	365
15.2	An HMM unfolded in time as a lattice (or trellis) showing all the possible trajectories.	368
15.3	Forward-backward procedure.	371
15.4	Computation of arc probabilities, $\xi_t(i, j)$.	375
15.5	Example of a left-to-right HMM.	381
16.1	Bayesian network modeling that rain is the cause of wet grass.	388
16.2	Head-to-tail connection.	390
16.3	Tail-to-tail connection.	391
16.4	Head-to-head connection.	392
16.5	Larger graphs are formed by combining simpler subgraphs over which information is propagated using the implied conditional independencies.	394
16.6	(a) Graphical model for classification. (b) Naive Bayes' classifier assumes independent inputs.	397
16.7	Hidden Markov model can be drawn as a graphical model where q^t are the hidden states and shaded O^t are observed.	398

16.8	Different types of HMM model different assumptions about the way the observed data (shown shaded) is generated from Markov sequences of latent variables.	399
16.9	Bayesian network for linear regression.	401
16.10	Examples of d-separation.	403
16.11	Inference along a chain.	404
16.12	In a tree, a node may have several children but a single parent.	406
16.13	In a polytree, a node may have several children and several parents, but the graph is singly connected; that is, there is a single chain between U_i and Y_j passing through X .	407
16.14	(a) A multiply connected graph, and (b) its corresponding junction tree with nodes clustered.	410
16.15	(a) A directed graph that would have a loop after moralization, and (b) its corresponding factor graph that is a tree.	412
16.16	Influence diagram corresponding to classification.	415
16.17	A dynamic version where we have a chain of graphs to show dependency in weather in consecutive days.	416
17.1	Base-learners are d_j and their outputs are combined using $f(\cdot)$.	424
17.2	AdaBoost algorithm.	432
17.3	Mixture of experts is a voting method where the votes, as given by the gating system, are a function of the input.	434
17.4	In stacked generalization, the combiner is another learner and is not restricted to being a linear combination as in voting.	436
17.5	Cascading is a multistage method where there is a sequence of classifiers, and the next one is used only when the preceding ones are not confident.	439
18.1	The agent interacts with an environment.	448
18.2	Value iteration algorithm for model-based learning.	453
18.3	Policy iteration algorithm for model-based learning.	454
18.4	Example to show that Q values increase but never decrease.	457
18.5	Q learning, which is an off-policy temporal difference algorithm.	458
18.6	Sarsa algorithm, which is an on-policy version of Q learning.	459
18.7	Example of an eligibility trace for a value.	460

18.8	Sarsa(λ) algorithm.	461
18.9	In the case of a partially observable environment, the agent has a state estimator (SE) that keeps an internal belief state b and the policy π generates actions based on the belief states.	465
18.10	Expected rewards and the effect of sensing in the Tiger problem.	468
18.11	Expected rewards change (a) if the hidden state can change, and (b) when we consider episodes of length two.	470
18.12	The grid world.	472
19.1	The process generates an output given an input and is affected by controllable and uncontrollable factors.	479
19.2	Different strategies of experimentation with two factors and five levels each.	480
19.3	(a) Typical ROC curve. (b) A classifier is preferred if its ROC curve is closer to the upper-left corner (larger AUC).	491
19.4	(a) Definition of precision and recall using Venn diagrams. (b) Precision is 1; all the retrieved records are relevant but there may be relevant ones not retrieved. (c) Recall is 1; all the relevant records are retrieved but there may also be irrelevant records that are retrieved.	492
19.5	95 percent of the unit normal distribution lies between -1.96 and 1.96 .	494
19.6	95 percent of the unit normal distribution lies before 1.64 .	496
A.1	Probability density function of Z , the unit normal distribution.	525

Tables

2.1	With two inputs, there are four possible cases and sixteen possible Boolean functions.	37
5.1	Reducing variance through simplifying assumptions.	100
11.1	Input and output for the AND function.	244
11.2	Input and output for the XOR function.	245
17.1	Classifier combination rules.	425
17.2	Example of combination rules on three learners and three classes.	425
19.1	Confusion matrix for two classes.	489
19.2	Performance measures used in two-class problems.	490
19.3	Type I error, type II error, and power of a test.	497
19.4	The analysis of variance (ANOVA) table for a single factor model.	507

Preface

Machine learning is programming computers to optimize a performance criterion using example data or past experience. We need learning in cases where we cannot directly write a computer program to solve a given problem, but need example data or experience. One case where learning is necessary is when human expertise does not exist, or when humans are unable to explain their expertise. Consider the recognition of spoken speech—that is, converting the acoustic speech signal to an ASCII text; we can do this task seemingly without any difficulty, but we are unable to explain how we do it. Different people utter the same word differently due to differences in age, gender, or accent. In machine learning, the approach is to collect a large collection of sample utterances from different people and learn to map these to words.

Another case is when the problem to be solved changes in time, or depends on the particular environment. We would like to have general-purpose systems that can adapt to their circumstances, rather than explicitly writing a different program for each special circumstance. Consider routing packets over a computer network. The path maximizing the quality of service from a source to destination changes continuously as the network traffic changes. A learning routing program is able to adapt to the best path by monitoring the network traffic. Another example is an intelligent user interface that can adapt to the biometrics of its user—namely, his or her accent, handwriting, working habits, and so forth.

Already, there are many successful applications of machine learning in various domains: There are commercially available systems for recognizing speech and handwriting. Retail companies analyze their past sales data to learn their customers' behavior to improve customer rela-

tionship management. Financial institutions analyze past transactions to predict customers' credit risks. Robots learn to optimize their behavior to complete a task using minimum resources. In bioinformatics, the huge amount of data can only be analyzed and knowledge extracted using computers. These are only some of the applications that we—that is, you and I—will discuss throughout this book. We can only imagine what future applications can be realized using machine learning: Cars that can drive themselves under different road and weather conditions, phones that can translate in real time to and from a foreign language, autonomous robots that can navigate in a new environment, for example, on the surface of another planet. Machine learning is certainly an exciting field to be working in!

The book discusses many methods that have their bases in different fields: statistics, pattern recognition, neural networks, artificial intelligence, signal processing, control, and data mining. In the past, research in these different communities followed different paths with different emphases. In this book, the aim is to incorporate them together to give a unified treatment of the problems and the proposed solutions to them.

This is an introductory textbook, intended for senior undergraduate and graduate-level courses on machine learning, as well as engineers working in the industry who are interested in the application of these methods. The prerequisites are courses on computer programming, probability, calculus, and linear algebra. The aim is to have all learning algorithms sufficiently explained so it will be a small step from the equations given in the book to a computer program. For some cases, pseudocode of algorithms are also included to make this task easier.

The book can be used for a one-semester course by sampling from the chapters, or it can be used for a two-semester course, possibly by discussing extra research papers; in such a case, I hope that the references at the end of each chapter are useful.

The Web page is <http://www.cmpe.boun.edu.tr/~ethem/i2ml/> where I will post information related to the book that becomes available after the book goes to press, for example, errata. I welcome your feedback via email to alpaydin@boun.edu.tr.

I very much enjoyed writing this book; I hope you will enjoy reading it.

Acknowledgments

The way you get good ideas is by working with talented people who are also fun to be with. The Department of Computer Engineering of Boğaziçi University is a wonderful place to work, and my colleagues gave me all the support I needed while working on this book. I would also like to thank my past and present students on whom I have field-tested the content that is now in book form.

While working on this book, I was supported by the Turkish Academy of Sciences, in the framework of the Young Scientist Award Program (EA-TÜBA-GEBİP/2001-1-1).

My special thanks go to Michael Jordan. I am deeply indebted to him for his support over the years and last for this book. His comments on the general organization of the book, and the first chapter, have greatly improved the book, both in content and form. Taner Bilgiç, Vladimir Cherkassky, Tom Dietterich, Fikret Gürgen, Olcay Taner Yıldız, and anonymous reviewers of the MIT Press also read parts of the book and provided invaluable feedback. I hope that they will sense my gratitude when they notice ideas that I have taken from their comments without proper acknowledgment. Of course, I alone am responsible for any errors or shortcomings.

My parents believe in me, and I am grateful for their enduring love and support. Sema Oktuğ is always there whenever I need her, and I will always be thankful for her friendship. I would also like to thank Hakan Ünlü for our many discussions over the years on several topics related to life, the universe, and everything.

This book is set using $\text{\LaTeX}$ macros prepared by Chris Manning for which I thank him. I would like to thank the editors of the Adaptive Computation and Machine Learning series, and Bob Prior, Valerie Geary, Kath-

leen Caruso, Sharon Deacon Warne, Erica Schultz, and Emily Gutheinz from the MIT Press for their continuous support and help during the completion of the book.

Notes for the Second Edition

Machine learning has seen important developments since the first edition appeared in 2004. First, application areas have grown rapidly. Internet-related technologies, such as search engines, recommendation systems, spam filters, and intrusion detection systems are now routinely using machine learning. In the field of bioinformatics and computational biology, methods that learn from data are being used more and more widely. In natural language processing applications—for example, machine translation—we are seeing a faster and faster move from programmed expert systems to methods that learn automatically from very large corpus of example text. In robotics, medical diagnosis, speech and image recognition, biometrics, finance, sometimes under the name pattern recognition, sometimes disguised as data mining, or under one of its many cloaks, we see more and more applications of the machine learning methods we discuss in this textbook.

Second, there have been supporting advances in theory. Especially, the idea of kernel functions and the kernel machines that use them allow a better representation of the problem and the associated convex optimization framework is one step further than multilayer perceptrons with sigmoid hidden units trained using gradient-descent. Bayesian methods through appropriately chosen prior distributions add expert knowledge to what the data tells us. Graphical models allow a representation as a network of interrelated nodes and efficient inference algorithms allow querying the network. It has thus become necessary that these three topics—namely, kernel methods, Bayesian estimation, and graphical models—which were sections in the first edition, be treated in more length, as three new chapters.

Another revelation hugely significant for the field has been in the real-

ization that machine learning experiments need to be designed better. We have gone a long way from using a single test set to methods for cross-validation to paired t tests. That is why, in this second edition, I have rewritten the chapter on statistical tests as one that includes the design and analysis of machine learning experiments. The point is that testing should not be a separate step done after all runs are completed (despite the fact that this new chapter is at the very end of the book); the whole process of experimentation should be designed beforehand, relevant factors defined, proper experimentation procedure decided upon, and then, and only then, the runs should be done and the results analyzed.

It has long been believed, especially by older members of the scientific community, that for machines to be as intelligent as us, that is, for artificial intelligence to be a reality, our current knowledge in general, or computer science in particular, is not sufficient. People largely are of the opinion that we need a new technology, a new type of material, a new type of computational mechanism or a new programming methodology, and that, until then, we can only “simulate” some aspects of human intelligence and only in a limited way but can never fully attain it.

I believe that we will soon prove them wrong. First we saw this in chess, and now we are seeing it in a whole variety of domains. Given enough memory and computation power, we can realize tasks with relatively simple algorithms; the trick here is learning, either learning from example data or learning from trial and error using reinforcement learning. It seems as if using supervised and mostly unsupervised learning algorithms—for example, machine translation—will soon be possible. The same holds for many other domains, for example, unmanned navigation in robotics using reinforcement learning. I believe that this will continue for many domains in artificial intelligence, and the key is learning. We do not need to come up with new algorithms if machines can learn themselves, assuming that we can provide them with enough data (not necessarily supervised) and computing power.

I would like to thank all the instructors and students of the first edition, from all over the world, including the reprint in India and the German translation. I am grateful to those who sent me words of appreciation and errata or who provided feedback in any other way. Please keep those emails coming. My email address is `alpaydin@boun.edu.tr`.

The second edition also provides more support on the Web. The book’s

Web site is <http://www.cmpe.boun.edu.tr/~ethem/i2m1>.

I would like to thank my past and present thesis students, Mehmet Gönen, Esma Kılıç, Murat Semerci, M. Aydın Ulaş, and Olcay Taner Yıldız, and also those who have taken CmpE 544, CmpE 545, CmpE 591, and CmpE 58E during these past few years. The best way to test your knowledge of a topic is by teaching it.

It has been a pleasure working with the MIT Press again on this second edition, and I thank Bob Prior, Ada Brunstein, Erin K. Shoudy, Kathleen Caruso, and Marcy Ross for all their help and support.

Notations

x	Scalar value
$\mathbf{x}$	Vector
$\mathbf{X}$	Matrix
$\mathbf{x}^T$	Transpose
$\mathbf{X}^{-1}$	Inverse
X	Random variable
$P(X)$	Probability mass function when X is discrete
$p(X)$	Probability density function when X is continuous
$P(X Y)$	Conditional probability of X given Y
$E[X]$	Expected value of the random variable X
$\text{Var}(X)$	Variance of X
$\text{Cov}(X, Y)$	Covariance of X and Y
$\text{Corr}(X, Y)$	Correlation of X and Y
μ	Mean
σ^2	Variance
Σ	Covariance matrix
m	Estimator to the mean
s^2	Estimator to the variance
$\mathbf{S}$	Estimator to the covariance matrix

$\mathcal{N}(\boldsymbol{\mu}, \sigma^2)$	Univariate normal distribution with mean $\boldsymbol{\mu}$ and variance σ^2
$\mathcal{Z}$	Unit normal distribution: $\mathcal{N}(0, 1)$
$\mathcal{N}_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	d -variate normal distribution with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$
x	Input
d	Number of inputs (input dimensionality)
y	Output
r	Required output
K	Number of outputs (classes)
N	Number of training instances
z	Hidden value, intrinsic dimension, latent factor
k	Number of hidden dimensions, latent factors
C_i	Class i
$\mathcal{X}$	Training sample
$\{x^t\}_{t=1}^N$	Set of x with index t ranging from 1 to N
$\{x^t, r^t\}_t$	Set of ordered pairs of input and desired output with index t
$g(x \theta)$	Function of x defined up to a set of parameters θ
$\arg \max_{\theta} g(x \theta)$	The argument θ for which g has its maximum value
$\arg \min_{\theta} g(x \theta)$	The argument θ for which g has its minimum value
$E(\theta \mathcal{X})$	Error function with parameters θ on the sample $\mathcal{X}$
$l(\theta \mathcal{X})$	Likelihood of parameters θ on the sample $\mathcal{X}$
$\mathcal{L}(\theta \mathcal{X})$	Log likelihood of parameters θ on the sample $\mathcal{X}$
$1(c)$	1 if c is true, 0 otherwise
$\#\{c\}$	Number of elements for which c is true
δ_{ij}	Kronecker delta: 1 if $i = j$, 0 otherwise

1

Introduction

1.1 What Is Machine Learning?

TO SOLVE a problem on a computer, we need an algorithm. An algorithm is a sequence of instructions that should be carried out to transform the input to output. For example, one can devise an algorithm for sorting. The input is a set of numbers and the output is their ordered list. For the same task, there may be various algorithms and we may be interested in finding the most efficient one, requiring the least number of instructions or memory or both.

For some tasks, however, we do not have an algorithm—for example, to tell spam emails from legitimate emails. We know what the input is: an email document that in the simplest case is a file of characters. We know what the output should be: a yes/no output indicating whether the message is spam or not. We do not know how to transform the input to the output. What can be considered spam changes in time and from individual to individual.

What we lack in knowledge, we make up for in data. We can easily compile thousands of example messages some of which we know to be spam and what we want is to “learn” what constitutes spam from them. In other words, we would like the computer (machine) to extract automatically the algorithm for this task. There is no need to learn to sort numbers, we already have algorithms for that; but there are many applications for which we do not have an algorithm but do have example data.

With advances in computer technology, we currently have the ability to store and process large amounts of data, as well as to access it from physically distant locations over a computer network. Most data acquisition

devices are digital now and record reliable data. Think, for example, of a supermarket chain that has hundreds of stores all over a country selling thousands of goods to millions of customers. The point of sale terminals record the details of each transaction: date, customer identification code, goods bought and their amount, total money spent, and so forth. This typically amounts to gigabytes of data every day. What the supermarket chain wants is to be able to predict who are the likely customers for a product. Again, the algorithm for this is not evident; it changes in time and by geographic location. The stored data becomes useful only when it is analyzed and turned into information that we can make use of, for example, to make predictions.

We do not know exactly which people are likely to buy this ice cream flavor, or the next book of this author, or see this new movie, or visit this city, or click this link. If we knew, we would not need any analysis of the data; we would just go ahead and write down the code. But because we do not, we can only collect data and hope to extract the answers to these and similar questions from data.

We do believe that there is a process that explains the data we observe. Though we do not know the details of the process underlying the generation of data—for example, consumer behavior—we know that it is not completely random. People do not go to supermarkets and buy things at random. When they buy beer, they buy chips; they buy ice cream in summer and spices for Glühwein in winter. There are certain patterns in the data.

We may not be able to identify the process completely, but we believe we can construct *a good and useful approximation*. That approximation may not explain everything, but may still be able to account for some part of the data. We believe that though identifying the complete process may not be possible, we can still detect certain patterns or regularities. This is the niche of machine learning. Such patterns may help us understand the process, or we can use those patterns to make predictions: Assuming that the future, at least the near future, will not be much different from the past when the sample data was collected, the future predictions can also be expected to be right.

Application of machine learning methods to large databases is called *data mining*. The analogy is that a large volume of earth and raw material is extracted from a mine, which when processed leads to a small amount of very precious material; similarly, in data mining, a large volume of data is processed to construct a simple model with valuable use,

for example, having high predictive accuracy. Its application areas are abundant: In addition to retail, in finance banks analyze their past data to build models to use in credit applications, fraud detection, and the stock market. In manufacturing, learning models are used for optimization, control, and troubleshooting. In medicine, learning programs are used for medical diagnosis. In telecommunications, call patterns are analyzed for network optimization and maximizing the quality of service. In science, large amounts of data in physics, astronomy, and biology can only be analyzed fast enough by computers. The World Wide Web is huge; it is constantly growing, and searching for relevant information cannot be done manually.

But machine learning is not just a database problem; it is also a part of artificial intelligence. To be intelligent, a system that is in a changing environment should have the ability to learn. If the system can learn and adapt to such changes, the system designer need not foresee and provide solutions for all possible situations.

Machine learning also helps us find solutions to many problems in vision, speech recognition, and robotics. Let us take the example of recognizing faces: This is a task we do effortlessly; every day we recognize family members and friends by looking at their faces or from their photographs, despite differences in pose, lighting, hair style, and so forth. But we do it unconsciously and are unable to explain how we do it. Because we are not able to explain our expertise, we cannot write the computer program. At the same time, we know that a face image is not just a random collection of pixels; a face has structure. It is symmetric. There are the eyes, the nose, the mouth, located in certain places on the face. Each person's face is a pattern composed of a particular combination of these. By analyzing sample face images of a person, a learning program captures the pattern specific to that person and then recognizes by checking for this pattern in a given image. This is one example of *pattern recognition*.

Machine learning is programming computers to optimize a performance criterion using example data or past experience. We have a model defined up to some parameters, and learning is the execution of a computer program to optimize the parameters of the model using the training data or past experience. The model may be *predictive* to make predictions in the future, or *descriptive* to gain knowledge from data, or both.

Machine learning uses the theory of statistics in building mathematical models, because the core task is making inference from a sample. The

role of computer science is twofold: First, in training, we need efficient algorithms to solve the optimization problem, as well as to store and process the massive amount of data we generally have. Second, once a model is learned, its representation and algorithmic solution for inference needs to be efficient as well. In certain applications, the efficiency of the learning or inference algorithm, namely, its space and time complexity, may be as important as its predictive accuracy.

Let us now discuss some example applications in more detail to gain more insight into the types and uses of machine learning.

1.2 Examples of Machine Learning Applications

1.2.1 Learning Associations

In the case of retail—for example, a supermarket chain—one application of machine learning is *basket analysis*, which is finding associations between products bought by customers: If people who buy X typically also buy Y , and if there is a customer who buys X and does not buy Y , he or she is a potential Y customer. Once we find such customers, we can target them for cross-selling.

ASSOCIATION RULE

In finding an *association rule*, we are interested in learning a conditional probability of the form $P(Y|X)$ where Y is the product we would like to condition on X , which is the product or the set of products which we know that the customer has already purchased.

Let us say, going over our data, we calculate that $P(\text{chips}|\text{beer}) = 0.7$. Then, we can define the rule:

70 percent of customers who buy beer also buy chips.

We may want to make a distinction among customers and toward this, estimate $P(Y|X, D)$ where D is the set of customer attributes, for example, gender, age, marital status, and so on, assuming that we have access to this information. If this is a bookseller instead of a supermarket, products can be books or authors. In the case of a Web portal, items correspond to links to Web pages, and we can estimate the links a user is likely to click and use this information to download such pages in advance for faster access.

1.2.2 Classification

A credit is an amount of money loaned by a financial institution, for example, a bank, to be paid back with interest, generally in installments. It is important for the bank to be able to predict in advance the risk associated with a loan, which is the probability that the customer will default and not pay the whole amount back. This is both to make sure that the bank will make a profit and also to not inconvenience a customer with a loan over his or her financial capacity.

In *credit scoring* (Hand 1998), the bank calculates the risk given the amount of credit and the information about the customer. The information about the customer includes data we have access to and is relevant in calculating his or her financial capacity—namely, income, savings, collaterals, profession, age, past financial history, and so forth. The bank has a record of past loans containing such customer data and whether the loan was paid back or not. From this data of particular applications, the aim is to infer a general rule coding the association between a customer's attributes and his risk. That is, the machine learning system fits a model to the past data to be able to calculate the risk for a new application and then decides to accept or refuse it accordingly.

CLASSIFICATION

This is an example of a *classification* problem where there are two classes: low-risk and high-risk customers. The information about a customer makes up the *input* to the classifier whose task is to assign the input to one of the two classes.

After training with the past data, a classification rule learned may be of the form

IF income > θ_1 AND savings > θ_2 THEN low-risk ELSE high-risk

DISCRIMINANT

for suitable values of θ_1 and θ_2 (see figure 1.1). This is an example of a *discriminant*; it is a function that separates the examples of different classes.

PREDICTION

Having a rule like this, the main application is *prediction*: Once we have a rule that fits the past data, if the future is similar to the past, then we can make correct predictions for novel instances. Given a new application with a certain income and savings, we can easily decide whether it is low-risk or high-risk.

In some cases, instead of making a 0/1 (low-risk/high-risk) type decision, we may want to calculate a probability, namely, $P(Y|X)$, where X are the customer attributes and Y is 0 or 1 respectively for low-risk

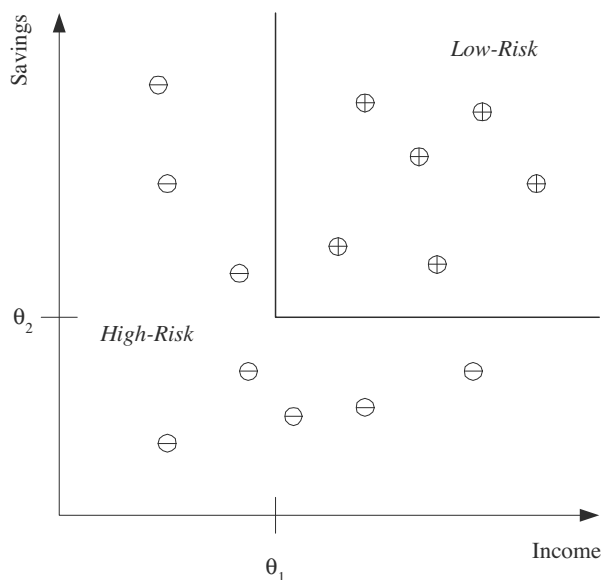


Figure 1.1 Example of a training dataset where each circle corresponds to one data instance with input values in the corresponding axes and its sign indicates the class. For simplicity, only two customer attributes, income and savings, are taken as input and the two classes are low-risk ('+') and high-risk ('-'). An example discriminant that separates the two types of examples is also shown.

and high-risk. From this perspective, we can see classification as learning an association from X to Y . Then for a given $X = x$, if we have $P(Y = 1|X = x) = 0.8$, we say that the customer has an 80 percent probability of being high-risk, or equivalently a 20 percent probability of being low-risk. We then decide whether to accept or refuse the loan depending on the possible gain and loss.

PATTERN
RECOGNITION

There are many applications of machine learning in *pattern recognition*. One is *optical character recognition*, which is recognizing character codes from their images. This is an example where there are multiple classes, as many as there are characters we would like to recognize. Especially interesting is the case when the characters are handwritten—for example, to read zip codes on envelopes or amounts on checks. People have different handwriting styles; characters may be written small or large, slanted, with a pen or pencil, and there are many possible images corresponding

to the same character. Though writing is a human invention, we do not have any system that is as accurate as a human reader. We do not have a formal description of ‘A’ that covers all ‘A’s and none of the non-‘A’s. Not having it, we take samples from writers and learn a definition of A-ness from these examples. But though we do not know what it is that makes an image an ‘A’, we are certain that all those distinct ‘A’s have something in common, which is what we want to extract from the examples. We know that a character image is not just a collection of random dots; it is a collection of strokes and has a regularity that we can capture by a learning program.

If we are reading a text, one factor we can make use of is the redundancy in human languages. A word is a *sequence* of characters and successive characters are not independent but are constrained by the words of the language. This has the advantage that even if we cannot recognize a character, we can still read the word. Such contextual dependencies may also occur in higher levels, between words and sentences, through the syntax and semantics of the language. There are machine learning algorithms to learn sequences and model such dependencies.

In the case of *face recognition*, the input is an image, the classes are people to be recognized, and the learning program should learn to associate the face images to identities. This problem is more difficult than optical character recognition because there are more classes, input image is larger, and a face is three-dimensional and differences in pose and lighting cause significant changes in the image. There may also be occlusion of certain inputs; for example, glasses may hide the eyes and eyebrows, and a beard may hide the chin.

In *medical diagnosis*, the inputs are the relevant information we have about the patient and the classes are the illnesses. The inputs contain the patient’s age, gender, past medical history, and current symptoms. Some tests may not have been applied to the patient, and thus these inputs would be missing. Tests take time, may be costly, and may inconvenience the patient so we do not want to apply them unless we believe that they will give us valuable information. In the case of a medical diagnosis, a wrong decision may lead to a wrong or no treatment, and in cases of doubt it is preferable that the classifier reject and defer decision to a human expert.

In *speech recognition*, the input is acoustic and the classes are words that can be uttered. This time the association to be learned is from an acoustic signal to a word of some language. Different people, because

of differences in age, gender, or accent, pronounce the same word differently, which makes this task rather difficult. Another difference of speech is that the input is *temporal*; words are uttered in time as a sequence of speech phonemes and some words are longer than others.

Acoustic information only helps up to a certain point, and as in optical character recognition, the integration of a “language model” is critical in speech recognition, and the best way to come up with a language model is again by learning it from some large corpus of example data. The applications of machine learning to *natural language processing* is constantly increasing. Spam filtering is one where spam generators on one side and filters on the other side keep finding more and more ingenious ways to outdo each other. Perhaps the most impressive would be *machine translation*. After decades of research on hand-coded translation rules, it has become apparent recently that the most promising way is to provide a very large number of example pairs of translated texts and have a program figure out automatically the rules to map one string of characters to another.

Biometrics is recognition or authentication of people using their physiological and/or behavioral characteristics that requires an integration of inputs from different modalities. Examples of physiological characteristics are images of the face, fingerprint, iris, and palm; examples of behavioral characteristics are dynamics of signature, voice, gait, and key stroke. As opposed to the usual identification procedures—photo, printed signature, or password—when there are many different (uncorrelated) inputs, forgeries (spoofing) would be more difficult and the system would be more accurate, hopefully without too much inconvenience to the users. Machine learning is used both in the separate recognizers for these different modalities and in the combination of their decisions to get an overall accept/reject decision, taking into account how reliable these different sources are.

KNOWLEDGE EXTRACTION

Learning a rule from data also allows *knowledge extraction*. The rule is a simple model that explains the data, and looking at this model we have an explanation about the process underlying the data. For example, once we learn the discriminant separating low-risk and high-risk customers, we have the knowledge of the properties of low-risk customers. We can then use this information to target potential low-risk customers more efficiently, for example, through advertising.

COMPRESSION

Learning also performs *compression* in that by fitting a rule to the data, we get an explanation that is simpler than the data, requiring less mem-

ory to store and less computation to process. Once you have the rules of addition, you do not need to remember the sum of every possible pair of numbers.

OUTLIER DETECTION

Another use of machine learning is *outlier detection*, which is finding the instances that do not obey the rule and are exceptions. In this case, after learning the rule, we are not interested in the rule but the exceptions not covered by the rule, which may imply anomalies requiring attention—for example, fraud.

1.2.3 Regression

REGRESSION

Let us say we want to have a system that can predict the price of a used car. Inputs are the car attributes—brand, year, engine capacity, mileage, and other information—that we believe affect a car’s worth. The output is the price of the car. Such problems where the output is a number are *regression* problems.

Let X denote the car attributes and Y be the price of the car. Again surveying the past transactions, we can collect a training data and the machine learning program fits a function to this data to learn Y as a function of X . An example is given in figure 1.2 where the fitted function is of the form

$$y = wx + w_0$$

for suitable values of w and w_0 .

SUPERVISED LEARNING

Both regression and classification are *supervised learning* problems where there is an input, X , an output, Y , and the task is to learn the mapping from the input to the output. The approach in machine learning is that we assume a model defined up to a set of parameters:

$$y = g(x|\theta)$$

where $g(\cdot)$ is the model and θ are its parameters. Y is a number in regression and is a class code (e.g., 0/1) in the case of classification. $g(\cdot)$ is the regression function or in classification, it is the discriminant function separating the instances of different classes. The machine learning program optimizes the parameters, θ , such that the approximation error is minimized, that is, our estimates are as close as possible to the correct values given in the training set. For example in figure 1.2, the model is linear and w and w_0 are the parameters optimized for best fit to the

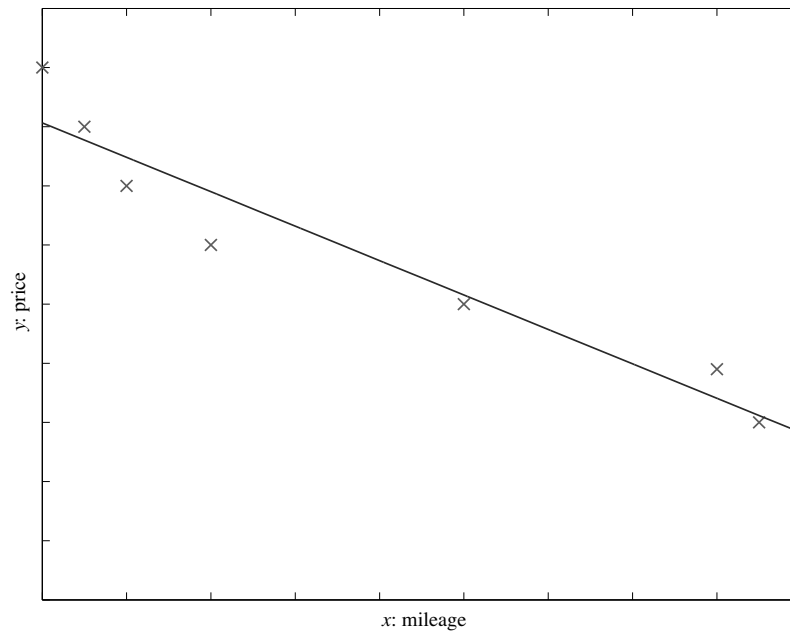


Figure 1.2 A training dataset of used cars and the function fitted. For simplicity, mileage is taken as the only input attribute and a linear model is used.

training data. In cases where the linear model is too restrictive, one can use for example a quadratic

$$y = w_2x^2 + w_1x + w_0$$

or a higher-order polynomial, or any other nonlinear function of the input, this time optimizing its parameters for best fit.

Another example of regression is navigation of a mobile robot, for example, an autonomous car, where the output is the angle by which the steering wheel should be turned at each time, to advance without hitting obstacles and deviating from the route. Inputs in such a case are provided by sensors on the car—for example, a video camera, GPS, and so forth. Training data can be collected by monitoring and recording the actions of a human driver.

One can envisage other applications of regression where one is trying

to optimize a function¹. Let us say we want to build a machine that roasts coffee. The machine has many inputs that affect the quality: various settings of temperatures, times, coffee bean type, and so forth. We make a number of experiments and for different settings of these inputs, we measure the quality of the coffee, for example, as consumer satisfaction. To find the optimal setting, we fit a regression model linking these inputs to coffee quality and choose new points to sample near the optimum of the current model to look for a better configuration. We sample these points, check quality, and add these to the data and fit a new model. This is generally called *response surface design*.

1.2.4 Unsupervised Learning

In supervised learning, the aim is to learn a mapping from the input to an output whose correct values are provided by a supervisor. In unsupervised learning, there is no such supervisor and we only have input data. The aim is to find the regularities in the input. There is a structure to the input space such that certain patterns occur more often than others, and we want to see what generally happens and what does not. In statistics, this is called *density estimation*.

DENSITY ESTIMATION
CLUSTERING

One method for density estimation is *clustering* where the aim is to find clusters or groupings of input. In the case of a company with a data of past customers, the customer data contains the demographic information as well as the past transactions with the company, and the company may want to see the distribution of the profile of its customers, to see what type of customers frequently occur. In such a case, a clustering model allocates customers similar in their attributes to the same group, providing the company with natural groupings of its customers; this is called *customer segmentation*. Once such groups are found, the company may decide strategies, for example, services and products, specific to different groups; this is known as *customer relationship management*. Such a grouping also allows identifying those who are outliers, namely, those who are different from other customers, which may imply a niche in the market that can be further exploited by the company.

An interesting application of clustering is in *image compression*. In this case, the input instances are image pixels represented as RGB values. A clustering program groups pixels with similar colors in the same

1. I would like to thank Michael Jordan for this example.

group, and such groups correspond to the colors occurring frequently in the image. If in an image, there are only shades of a small number of colors, and if we code those belonging to the same group with one color, for example, their average, then the image is quantized. Let us say the pixels are 24 bits to represent 16 million colors, but if there are shades of only 64 main colors, for each pixel we need 6 bits instead of 24. For example, if the scene has various shades of blue in different parts of the image, and if we use the same average blue for all of them, we lose the details in the image but gain space in storage and transmission. Ideally, one would like to identify higher-level regularities by analyzing repeated image patterns, for example, texture, objects, and so forth. This allows a higher-level, simpler, and more useful description of the scene, and for example, achieves better compression than compressing at the pixel level. If we have scanned document pages, we do not have random on/off pixels but bitmap images of characters. There is structure in the data, and we make use of this redundancy by finding a shorter description of the data: 16×16 bitmap of 'A' takes 32 bytes; its ASCII code is only 1 byte.

In *document clustering*, the aim is to group similar documents. For example, news reports can be subdivided as those related to politics, sports, fashion, arts, and so on. Commonly, a document is represented as a *bag of words*, that is, we predefine a lexicon of N words and each document is an N -dimensional binary vector whose element i is 1 if word i appears in the document; suffixes “-s” and “-ing” are removed to avoid duplicates and words such as “of,” “and,” and so forth, which are not informative, are not used. Documents are then grouped depending on the number of shared words. It is of course here critical how the lexicon is chosen.

Machine learning methods are also used in *bioinformatics*. DNA in our genome is the “blueprint of life” and is a sequence of bases, namely, A, G, C, and T. RNA is transcribed from DNA, and proteins are translated from the RNA. Proteins are what the living body is and does. Just as a DNA is a sequence of bases, a protein is a sequence of amino acids (as defined by bases). One application area of computer science in molecular biology is *alignment*, which is matching one sequence to another. This is a difficult string matching problem because strings may be quite long, there are many template strings to match against, and there may be deletions, insertions, and substitutions. Clustering is used in learning *motifs*, which are sequences of amino acids that occur repeatedly in proteins. Motifs are of interest because they may correspond to structural or functional

elements within the sequences they characterize. The analogy is that if the amino acids are letters and proteins are sentences, motifs are like words, namely, a string of letters with a particular meaning occurring frequently in different sentences.

1.2.5 Reinforcement Learning

REINFORCEMENT LEARNING

In some applications, the output of the system is a sequence of *actions*. In such a case, a single action is not important; what is important is the *policy* that is the sequence of correct actions to reach the goal. There is no such thing as the best action in any intermediate state; an action is good if it is part of a good policy. In such a case, the machine learning program should be able to assess the goodness of policies and learn from past good action sequences to be able to generate a policy. Such learning methods are called *reinforcement learning* algorithms.

A good example is *game playing* where a single move by itself is not that important; it is the sequence of right moves that is good. A move is good if it is part of a good game playing policy. Game playing is an important research area in both artificial intelligence and machine learning. This is because games are easy to describe and at the same time, they are quite difficult to play well. A game like chess has a small number of rules but it is very complex because of the large number of possible moves at each state and the large number of moves that a game contains. Once we have good algorithms that can learn to play games well, we can also apply them to applications with more evident economic utility.

A robot navigating in an environment in search of a goal location is another application area of reinforcement learning. At any time, the robot can move in one of a number of directions. After a number of trial runs, it should learn the correct sequence of actions to reach to the goal state from an initial state, doing this as quickly as possible and without hitting any of the obstacles.

One factor that makes reinforcement learning harder is when the system has unreliable and partial sensory information. For example, a robot equipped with a video camera has incomplete information and thus at any time is in a *partially observable state* and should decide taking into account this uncertainty; for example, it may not know its exact location in a room but only that there is a wall to its left. A task may also require a concurrent operation of *multiple agents* that should interact and

cooperate to accomplish a common goal. An example is a team of robots playing soccer.

1.3 Notes

Evolution is the major force that defines our bodily shape as well as our built-in instincts and reflexes. We also learn to change our behavior during our lifetime. This helps us cope with changes in the environment that cannot be predicted by evolution. Organisms that have a short life in a well-defined environment may have all their behavior built-in, but instead of hardwiring into us all sorts of behavior for any circumstance that we could encounter in our life, evolution gave us a large brain and a mechanism to learn, such that we could update ourselves with experience and adapt to different environments. When we learn the best strategy in a certain situation, that knowledge is stored in our brain, and when the situation arises again, when we re-cognize (“cognize” means to know) the situation, we can recall the suitable strategy and act accordingly. Learning has its limits though; there may be things that we can never learn with the limited capacity of our brains, just like we can never “learn” to grow a third arm, or an eye on the back of our head, even if either would be useful. See Leahey and Harris 1997 for learning and cognition from the point of view of psychology. Note that unlike in psychology, cognitive science, or neuroscience, our aim in machine learning is not to understand the processes underlying learning in humans and animals, but to build useful systems, as in any domain of engineering.

Almost all of science is fitting models to data. Scientists design experiments and make observations and collect data. They then try to extract knowledge by finding out simple models that explain the data they observed. This is called *induction* and is the process of extracting general rules from a set of particular cases.

We are now at a point that such analysis of data can no longer be done by people, both because the amount of data is huge and because people who can do such analysis are rare and manual analysis is costly. There is thus a growing interest in computer models that can analyze data and extract information automatically from them, that is, learn.

The methods we are going to discuss in the coming chapters have their origins in different scientific domains. Sometimes the same algorithm

was independently invented in more than one field, following a different historical path.

In statistics, going from particular observations to general descriptions is called *inference* and learning is called *estimation*. Classification is called *discriminant analysis* in statistics (McLachlan 1992; Hastie, Tibshirani, and Friedman 2001). Before computers were cheap and abundant, statisticians could only work with small samples. Statisticians, being mathematicians, worked mostly with simple parametric models that could be analyzed mathematically. In engineering, classification is called *pattern recognition* and the approach is nonparametric and much more empirical (Duda, Hart, and Stork 2001; Webb 1999). Machine learning is related to *artificial intelligence* (Russell and Norvig 2002) because an intelligent system should be able to adapt to changes in its environment. Application areas like vision, speech, and robotics are also tasks that are best learned from sample data. In electrical engineering, research in *signal processing* resulted in adaptive computer vision and speech programs. Among these, the development of *hidden Markov models* (HMM) for speech recognition is especially important.

In the late 1980s with advances in VLSI technology and the possibility of building parallel hardware containing thousands of processors, the field of *artificial neural networks* was reinvented as a possible theory to distribute computation over a large number of processing units (Bishop 1995). Over time, it has been realized in the neural network community that most neural network learning algorithms have their basis in statistics—for example, the multilayer perceptron is another class of nonparametric estimator—and claims of brainlike computation have started to fade.

In recent years, kernel-based algorithms, such as support vector machines, have become popular, which, through the use of kernel functions, can be adapted to various applications, especially in bioinformatics and language processing. It is common knowledge nowadays that a good representation of data is critical for learning and kernel functions turn out to be a very good way to introduce such expert knowledge.

Recently, with the reduced cost of storage and connectivity, it has become possible to have very large datasets available over the Internet, and this, coupled with cheaper computation, have made it possible to run learning algorithms on a lot of data. In the past few decades, it was generally believed that for artificial intelligence to be possible, we needed a new paradigm, a new type of thinking, a new model of computation

or a whole new set of algorithms. Taking into account the recent successes in machine learning in various domains, it may be claimed that what we needed was not new algorithms but a lot of example data and sufficient computing power to run the algorithms on that much data. For example, the roots of support vector machines go to potential functions, linear classifiers, and neighbor-based methods, proposed in the 1950s or the 1960s; it is just that we did not have fast computers or large storage then for these algorithms to show their full potential. It may be conjectured that tasks such as machine translation, and even planning, can be solved with such relatively simple learning algorithms but trained on large amounts of example data, or through long runs of trial and error. Intelligence seems not to originate from some outlandish formula, but rather from the patient, almost brute-force use of a simple, straightforward algorithm.

Data mining is the name coined in the business world for the application of machine learning algorithms to large amounts of data (Witten and Frank 2005; Han and Kamber 2006). In computer science, it used to be called *knowledge discovery in databases* (KDD).

Research in these different communities (statistics, pattern recognition, neural networks, signal processing, control, artificial intelligence, and data mining) followed different paths in the past with different emphases. In this book, the aim is to incorporate these emphases together to give a unified treatment of the problems and the proposed solutions to them.

1.4 Relevant Resources

The latest research on machine learning is distributed over journals and conferences from different fields. Dedicated journals are *Machine Learning* and *Journal of Machine Learning Research*. Journals with a neural network emphasis are *Neural Computation*, *Neural Networks*, and the *IEEE Transactions on Neural Networks*. Statistics journals like *Annals of Statistics* and *Journal of the American Statistical Association* also publish machine learning papers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* is another source.

Journals on artificial intelligence, pattern recognition, fuzzy logic, and signal processing also contain machine learning papers. Journals with an emphasis on data mining are *Data Mining and Knowledge Discovery*, *IEEE*

Transactions on Knowledge and Data Engineering, and *ACM Special Interest Group on Knowledge Discovery and Data Mining Explorations Journal*.

The major conferences on machine learning are *Neural Information Processing Systems* (NIPS), *Uncertainty in Artificial Intelligence* (UAI), *International Conference on Machine Learning* (ICML), *European Conference on Machine Learning* (ECML), and *Computational Learning Theory* (COLT). *International Joint Conference on Artificial Intelligence* (IJCAI), as well as conferences on neural networks, pattern recognition, fuzzy logic, and genetic algorithms, have sessions on machine learning and conferences on application areas like computer vision, speech technology, robotics, and data mining.

There are a number of dataset repositories on the Internet that are used frequently by machine learning researchers for benchmarking purposes:

- *UCI Repository* for machine learning is the most popular repository:
<http://www.ics.uci.edu/~mlearn/MLRepository.html>
- *UCI KDD Archive*:
<http://kdd.ics.uci.edu/summary.data.application.html>
- *Statlib*: <http://lib.stat.cmu.edu>
- *Delve*: <http://www.cs.utoronto.ca/~delve/>

In addition to these, there are also repositories for particular applications, for example, computational biology, face recognition, speech recognition, and so forth.

New and larger datasets are constantly being added to these repositories, especially to the UCI repository. Still, some researchers believe that such repositories do not reflect the full characteristics of real data and are of limited scope, and therefore accuracies on datasets from such repositories are not indicative of anything. It may even be claimed that when some datasets from a fixed repository are used repeatedly while tailoring a new algorithm, we are generating a new set of “UCI algorithms” specialized for those datasets.

As we will see in later chapters, different algorithms are better on different tasks anyway, and therefore it is best to keep one application in mind, to have one or a number of large datasets drawn for that and compare algorithms on those, for that specific task.

Most recent papers by machine learning researchers are accessible over the Internet, and a good place to start searching is the NEC Research In-

dex at <http://citeseer.ist.psu.edu>. Most authors also make codes of their algorithms available over the Web. There are also free software packages implementing various machine learning algorithms, and among these, Weka is especially noteworthy: <http://www.cs.waikato.ac.nz/ml/weka/>.

1.5 Exercises

1. Imagine you have two possibilities: You can fax a document, that is, send the image, or you can use an optical character reader (OCR) and send the text file. Discuss the advantage and disadvantages of the two approaches in a comparative manner. When would one be preferable over the other?
2. Let us say we are building an OCR and for each character, we store the bitmap of that character as a template that we match with the read character pixel by pixel. Explain when such a system would fail. Why are barcode readers still used?
3. Assume we are given the task to build a system that can distinguish junk e-mail. What is in a junk e-mail that lets us know that it is junk? How can the computer detect junk through a syntactic analysis? What would you like the computer to do if it detects a junk e-mail—delete it automatically, move it to a different file, or just highlight it on the screen?
4. Let us say you are given the task of building an automated taxi. Define the constraints. What are the inputs? What is the output? How can you communicate with the passenger? Do you need to communicate with the other automated taxis, that is, do you need a “language”?
5. In basket analysis, we want to find the dependence between two items X and Y . Given a database of customer transactions, how can you find these dependencies? How would you generalize this to more than two items?
6. How can you predict the next command to be typed by the user? Or the next page to be downloaded over the Web? When would such a prediction be useful? When would it be annoying?
7. In your everyday newspaper, find five sample news reports for each category of politics, sports, and the arts. Go over these reports and find words that are used frequently for each category, which may help us discriminate between different categories. For example, a news report on politics is likely to include words such as “government,” “recession,” “congress,” and so forth, whereas a news report on the arts may include “album,” “canvas,” or “theater.” There are also words such as “goal” that are ambiguous.
8. If a face image is a 100×100 image, written in row-major, this is a 10,000-dimensional vector. If we shift the image one pixel to the right, this will be a

very different vector in the 10,000-dimensional space. How can we build face recognizers robust to such distortions?

9. Take a word, for example, “machine.” Write it ten times. Also ask a friend to write it ten times. Analyzing these twenty images, try to find features, types of strokes, curvatures, loops, how you make the dots, and so on, that discriminate your handwriting from your friend’s.
10. In estimating the price of a used car, rather than estimating the absolute price it makes more sense to estimate the percent depreciation over the original price. Why?

1.6 References

- Bishop, C. M. 1995. *Neural Networks for Pattern Recognition*. Oxford: Oxford University Press.
- Duda, R. O., P. E. Hart, and D. G. Stork. 2001. *Pattern Classification*, 2nd ed. New York: Wiley.
- Han, J., and M. Kamber. 2006. *Data Mining: Concepts and Techniques*, 2nd ed. San Francisco: Morgan Kaufmann.
- Hand, D. J. 1998. “Consumer Credit and Statistics.” In *Statistics in Finance*, ed. D. J. Hand and S. D. Jacka, 69–81. London: Arnold.
- Hastie, T., R. Tibshirani, and J. Friedman. 2001. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. New York: Springer.
- Leahey, T. H., and R. J. Harris. 1997. *Learning and Cognition*, 4th ed. New York: Prentice Hall.
- McLachlan, G. J. 1992. *Discriminant Analysis and Statistical Pattern Recognition*. New York: Wiley.
- Russell, S., and P. Norvig. 2002. *Artificial Intelligence: A Modern Approach*, 2nd ed. New York: Prentice Hall.
- Webb, A. 1999. *Statistical Pattern Recognition*. London: Arnold.
- Witten, I. H., and E. Frank. 2005. *Data Mining: Practical Machine Learning Tools and Techniques*, 2nd ed. San Francisco: Morgan Kaufmann.